Divisão e Conquista *

PCC104 – Projeto e ANálise de Algoritmos

9 de abril de 2026

Sumário

1 Introdução **1**
1.1 Exemplo clássico - MergeSort . . . 1
1.2 Análise de Complexidade do Procedimento Merge . . . 4

2 Recorrências **6**

3 Indução Matemática **6**
3.1 Estrutura da Prova por Indução . . . 6
3.2 Exemplo: Soma dos Primeiros n Naturais . . . 6
3.3 Indução Forte . . . 7
3.4 Exemplo com Indução Forte . . . 7
3.5 Indução e Recorrências . . . 7

4 Método de Substituição **7**
4.1 Estrutura do Método . . . 7
4.2 Exemplo: Recorrência do Merge-Sort . . . 7
4.3 Flexibilidade na Escolha do Caso Base . . . 8
4.3.1 Por que a Falha em $n = 1$ Não É Problema . . . 8
4.3.2 A Justificativa Formal . . . 8
4.3.3 A Observação do Cormen . . . 8
4.4 Armadilhas Comuns . . . 8
4.5 Como Obter Boas Conjecturas . . . 8

5 Método da Árvore de Recursão **8**
5.1 Construção da Árvore . . . 9
5.2 Exemplo: Recorrência do Merge-Sort . . . 9
5.3 Exemplo: Recorrência com Subproblemas Desiguais . . . 9
5.4 Exemplo: Custo Geométrico por Nível . . . 10
5.5 Árvore de Recursão como Conjectura . . . 11

6 Método da Árvore de Recursão **11**
6.1 Construção da Árvore . . . 11
6.2 Exemplo: Recorrência do Merge-Sort . . . 11
6.3 Exemplo: Recorrência com Subproblemas Desiguais . . . 12
6.4 Exemplo: Custo Geométrico por Nível . . . 12
6.5 Árvore de Recursão como Conjectura . . . 12

7 Método mestre **12**

1 Introdução

Definição: Divisão e conquista é uma técnica de projeto de algoritmos que resolve problemas de forma recursiva, seguindo três etapas:

1. **Dividir:** o problema original é dividido em subproblemas menores, que são instâncias menores do mesmo problema.
2. **Conquistar:** os subproblemas são resolvidos recursivamente. Quando o subproblema é suficientemente pequeno, ele é resolvido diretamente (caso base).
3. **Combinar:** as soluções dos subproblemas são combinadas para construir a solução do problema original.

1.1 Exemplo clássico - MergeSort

Merge Sort: o arranjo é dividido ao meio, cada metade é ordenada recursivamente e, ao final, as duas metades ordenadas são combinadas em um único arranjo ordenado.

*Material baseado em (Cormen et al., 2022)

1. **Dividir:** o arranjo de n elementos é dividido em dois subarranjos de $n/2$ elementos cada.
2. **Conquistar:** os dois subarranjos são ordenados recursivamente usando o próprio Merge Sort. Quando o subarranjo tem tamanho 1, ele já está trivialmente ordenado, esse é o caso base.
3. **Combinar:** os dois subarranjos ordenados são intercalados (*merged*) para produzir um único arranjo ordenado. Essa é a etapa central do algoritmo, onde ocorre o trabalho real.

Comecemos pelo Combinar:

Algorithm 1 Merge(A, p, q, r)

```
n_L = q − p + 1                                  ▷ tamanho do subarranjo esquerdo
n_R = r − q                                      ▷ tamanho do subarranjo direito
seja L[0 .. n_L − 1] e R[0 .. n_R − 1] novos arranjos
for i = 0 to n_L − 1 do
    L[i] = A[p + i]
end for
for j = 0 to n_R − 1 do
    R[j] = A[q + j + 1]
end for
i = 0; j = 0; k = p
while i < n_L and j < n_R do
    if L[i] ≤ R[j] then
        A[k] = L[i]; i = i + 1
    else
        A[k] = R[j]; j = j + 1
    end if
    k = k + 1
end while
while i < n_L do
    A[k] = L[i]; i = i + 1; k = k + 1
end while
while j < n_R do
    A[k] = R[j]; j = j + 1; k = k + 1
end while
```

Exemplo de execução do Merge:

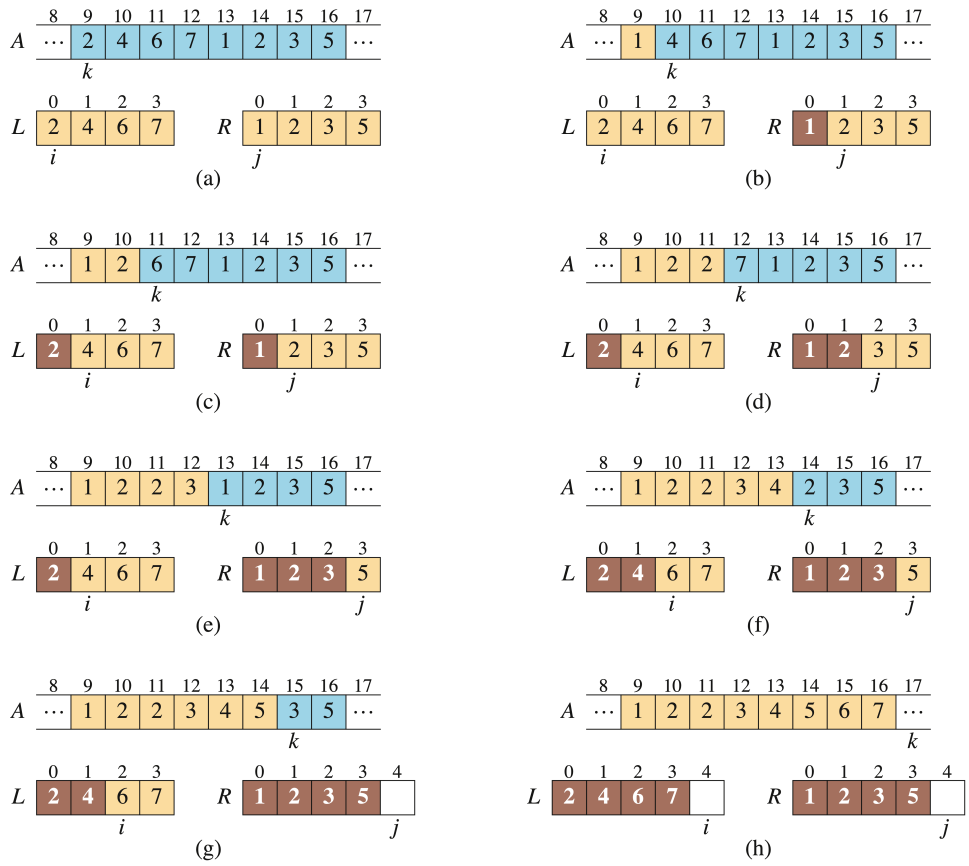


Figura 1: Exemplo de execução do método Merge

Agora o método MergeSort

Algorithm 2 MergeSort(A, p, r)

```
if p ≥ r then
    return
end if
q = ⌊(p + r)/2⌋
MergeSort(A, p, q)
MergeSort(A, q + 1, r)
Merge(A, p, q, r)
```

Exemplo de execução do Merge:

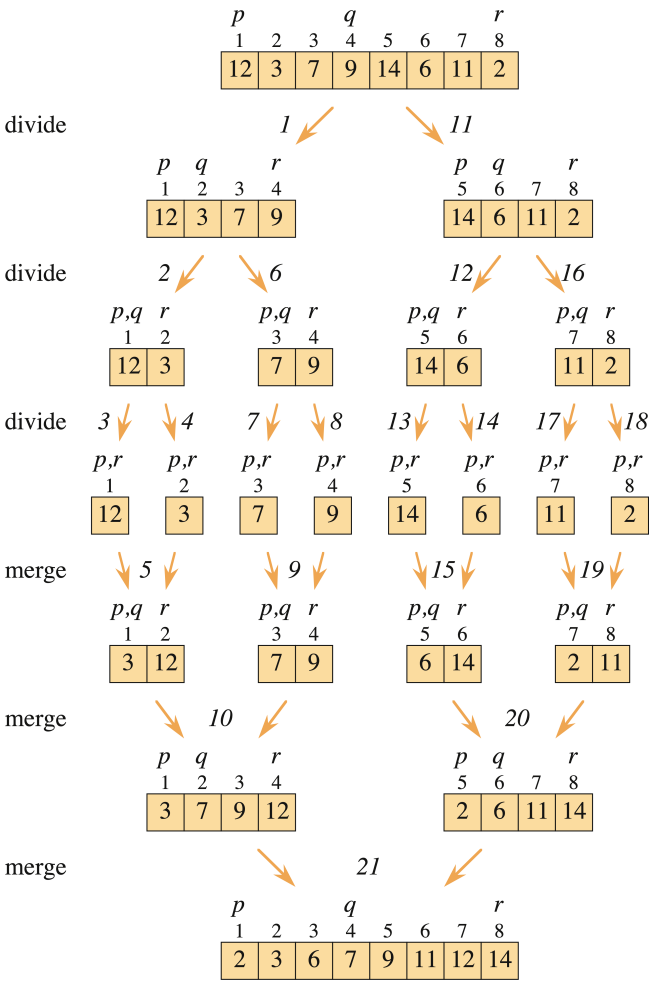


Figura 2: Exemplo de execução do método MergeSort

1.2 Análise de Complexidade do Procedimento Merge

Analisamos o custo de cada linha do procedimento $\textsc{Merge}(A, p, q, r)$, onde $n = r - p + 1$ denota o número total de elementos no subarranjo $A[p\,..\,r]$ sendo intercalado, com $n_L = q - p + 1$ e $n_R = r - q$.

Linha	Instrução	Custo	Vezes
1	$n_L = q - p + 1$	c_1	1
2	$n_R = r - q$	c_2	1
3	aloca $L[0\,..\,n_L - 1]$ e $R[0\,..\,n_R - 1]$	c_3	1
4	**for** $i = 0$ **to** $n_L - 1$	c_4	$\sum_{i=0}^{n_L} 1$
5	$\quad L[i] = A[p + i]$	c_5	$\sum_{i=0}^{n_L - 1} 1$
6	**for** $j = 0$ **to** $n_R - 1$	c_6	$\sum_{j=0}^{n_R} 1$
7	$\quad R[j] = A[q + j + 1]$	c_7	$\sum_{j=0}^{n_R - 1} 1$
8	$i = 0;\ \ j = 0;\ \ k = p$	c_8	1
9	**while** $i < n_L$ **and** $j < n_R$	c_9	$\sum_{k=p}^{p+t} 1$
10	$\quad$ **if** $L[i] \leq R[j]$	c_{10}	$\sum_{k=p}^{p+t-1} 1$
11	$\quad\quad A[k] = L[i];\ \ i = i + 1$	c_{11}	t_L
12	$\quad$ **else**		
13	$\quad\quad A[k] = R[j];\ \ j = j + 1$	c_{12}	t_R
14	$\quad k = k + 1$	c_{13}	$\sum_{k=p}^{p+t-1} 1$
15	**while** $i < n_L$	c_{14}	$\sum_{i=t_L}^{n_L} 1$
16	$\quad A[k] = L[i];\ \ i{=}i{+}1;\ \ k{=}k{+}1$	c_{15}	$\sum_{i=t_L}^{n_L - 1} 1$
17	**while** $j < n_R$	c_{16}	$\sum_{j=t_R}^{n_R} 1$
18	$\quad A[k] = R[j];\ \ j{=}j{+}1;\ \ k{=}k{+}1$	c_{17}	$\sum_{j=t_R}^{n_R - 1} 1$

onde t é o número de iterações do laço **while** principal (linha 9), t_L é o número de vezes que um elemento de L é copiado nesse laço, e $t_R = t - t_L$ é o número de vezes que um elemento de R é copiado. Note que $t_L + t_R = t$.

O tempo total de execução é dado por:

$$\begin{aligned} T(n) = {} & c_1 + c_2 + c_3 + c_4(n_L + 1) + c_5 n_L + c_6(n_R + 1) + c_7 n_R + c_8 \\ & + c_9(t + 1) + c_{10}\, t + c_{11}\, t_L + c_{12}\, t_R + c_{13}\, t \\ & + c_{14}(n_L - t_L + 1) + c_{15}(n_L - t_L) \\ & + c_{16}(n_R - t_R + 1) + c_{17}(n_R - t_R) \end{aligned}$$

Observamos que cada um dos n elementos é copiado para A exatamente uma vez, seja no laço principal (linhas 11 ou 13), seja nos laços de finalização (linhas 16 ou 18). Portanto:

$$t_L + (n_L - t_L) + t_R + (n_R - t_R) = n_L + n_R = n$$

Agrupando os termos e simplificando, obtemos:

$$T(n) = \alpha \cdot n + \beta$$

onde α e β são constantes que dependem dos custos c_i. Portanto:

$$T(n) = \Theta(n)$$

Concluímos que o procedimento Merge executa em tempo linear no número de elementos sendo intercalados. Este resultado é fundamental para a análise do Merge-Sort, cuja recorrência

$$T(n) = 2\,T(n/2) + \Theta(n)$$

resolve-se, pelo Teorema Mestre (caso 2, com $a = 2$, $b = 2$ e $f(n) = \Theta(n)$), para $T(n) = \Theta(n \lg n)$.

2 Recorrências

Uma **recorrência** é uma equação ou desigualdade que descreve uma função $T(n)$ em termos de seu valor em entradas menores. Recorrências surgem naturalmente quando um algoritmo divide um problema de tamanho n em subproblemas menores, resolve-os recursivamente e combina as soluções.

Uma recorrência é composta por dois elementos essenciais:

O **caso recursivo** expressa $T(n)$ em função de valores $T(m)$ com $m < n$. Por exemplo, na recorrência do Merge-Sort, $T(n) = 2\,T(n/2) + \Theta(n)$ para $n > 1$, o caso recursivo captura o custo de resolver dois subproblemas de tamanho $n/2$ e intercalar os resultados em tempo $\Theta(n)$.

O **caso base** (ou **condição de contorno**) fornece o valor de $T(n)$ para entradas suficientemente pequenas, encerrando a recursão. No mesmo exemplo, $T(1) = \Theta(1)$, pois ordenar um único elemento tem custo constante.

Uma recorrência é **bem definida** quando todo valor de $T(n)$ pode ser determinado de forma não ambígua: o caso recursivo sempre reduz o argumento em direção ao caso base, e o caso base está definido. Exemplos clássicos:

$$T(n) = T(n-1) + c, \quad T(1) = c$$

que descreve a busca linear e resolve-se para $T(n) = \Theta(n)$; e

$$T(n) = 2\,T(n/2) + \Theta(n), \quad T(1) = \Theta(1)$$

que descreve o Merge-Sort e resolve-se para $T(n) = \Theta(n \lg n)$.

Uma recorrência é **mal definida** quando o valor de $T(n)$ não pode ser determinado univocamente. Isso acontece, por exemplo, quando falta o caso base, como em $T(n) = 2\,T(n/2) + n$ sem nenhuma condição de contorno — a solução fica indeterminada a menos de uma constante multiplicativa. Outro caso é quando o argumento não decresce em direção ao caso base, como $T(n) = T(n) + 1$, que não reduz o tamanho do problema e portanto não converge. Também pode ocorrer quando o domínio é ambíguo: a recorrência $T(n) = 2\,T(n/2) + n$ aplicada a n ímpar requer uma convenção sobre o significado de $T(n/2)$ — o Cormen resolve isso usando pisos e tetos, escrevendo $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n)$, e depois mostra que essa distinção não afeta a ordem assintótica.

3 Indução Matemática

A **indução matemática** é uma técnica de prova utilizada para demonstrar que uma proposição $P(n)$ vale para todo número natural $n \geq n_0$, onde n_0 é um valor base. O método de substituição para resolver recorrências, apresentado na seção seguinte, depende diretamente desta técnica, de modo que sua compreensão é pré-requisito essencial.

3.1 Estrutura da Prova por Indução

Uma prova por indução é composta por dois passos:

1. **Caso base.** Verificar que $P(n_0)$ é verdadeira para algum valor inicial n_0.
2. **Passo indutivo.** Assumir que $P(k)$ é verdadeira para algum $k \geq n_0$ (esta suposição é chamada de **hipótese indutiva**) e, a partir dela, provar que $P(k+1)$ também é verdadeira.

Se ambos os passos são satisfeitos, concluímos que $P(n)$ vale para todo $n \geq n_0$. A intuição é a de um efeito dominó: o caso base derruba a primeira peça, e o passo indutivo garante que cada peça derruba a seguinte.

3.2 Exemplo: Soma dos Primeiros n Naturais

Demonstremos que, para todo $n \geq 1$,

$$P(n): \quad \sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

Caso base. Para $n = 1$, o lado esquerdo vale $\sum_{i=1}^{1} i = 1$ e o lado direito vale $1 \cdot 2/2 = 1$. Logo $P(1)$ é verdadeira.

Passo indutivo. Suponha que $P(k)$ é verdadeira, isto é, $\sum_{i=1}^{k} i = k(k+1)/2$. Queremos provar que $P(k+1)$ vale. Temos:

$$\begin{aligned}
\sum_{i=1}^{k+1} i &= \left(\sum_{i=1}^{k} i\right) + (k+1) \\
&= \frac{k(k+1)}{2} + (k+1) \\
&= \frac{k(k+1) + 2(k+1)}{2} \\
&= \frac{(k+1)(k+2)}{2}
\end{aligned}$$

que é exatamente a fórmula avaliada em $n = k+1$. Portanto $P(k+1)$ é verdadeira, e por indução $P(n)$ vale para todo $n \geq 1$. $\square$

3.3 Indução Forte

Na **indução forte** (ou **indução completa**), a hipótese indutiva é mais poderosa: em vez de assumir apenas que $P(k)$ vale, assumimos que $P(m)$ vale para *todo* m com $n_0 \leq m \leq k$, e a partir disso provamos $P(k+1)$.

Essa variante é logicamente equivalente à indução simples, mas em muitas situações é mais conveniente. Em particular, o método de substituição para recorrências utiliza indução forte, pois o valor de $T(n)$ pode depender de T avaliada em múltiplas entradas menores — como $T(\lfloor n/2 \rfloor)$ e $T(\lceil n/2 \rceil)$ simultaneamente — e precisamos que a hipótese valha para todas elas, não apenas para $T(n-1)$.

3.4 Exemplo com Indução Forte

Demonstremos que, para todo $n \geq 1$, todo inteiro $n \geq 2$ pode ser escrito como produto de números primos.

Caso base. Para $n = 2$, o próprio 2 é primo, logo é trivialmente um produto de primos.

Passo indutivo. Suponha que, para todo m com $2 \leq m \leq k$, m pode ser escrito como produto de primos. Considere $k+1$. Se $k+1$ é primo, a afirmação é imediata. Se $k+1$ não é primo, então existem inteiros a, b com $2 \leq a, b \leq k$ tais que $k+1 = a \cdot b$. Pela hipótese indutiva (que vale para todo $m \leq k$), ambos a e b podem ser escritos como produto de primos, e portanto $k+1 = a \cdot b$ também pode. □

Note que a indução simples não seria suficiente aqui: saber que k é produto de primos nada diz sobre a fatoração de $k+1$.

3.5 Indução e Recorrências

A conexão entre indução e recorrências é direta. Provar que $T(n) \leq c\,g(n)$ para uma recorrência é essencialmente uma prova por indução forte: assumimos que $T(m) \leq c\,g(m)$ para todo $m < n$, substituímos na recorrência, e verificamos que $T(n) \leq c\,g(n)$ segue algebricamente. O caso base corresponde a verificar a desigualdade para valores pequenos de n, possivelmente escolhendo a constante c grande o suficiente para acomodá-los. Esta é precisamente a mecânica do **método de substituição**, que será desenvolvido na próxima seção.

4 Método de Substituição

O **método de substituição** para resolver recorrências, conforme apresentado no Cormen (CLRS), consiste em dois passos: (1) conjecturar a forma da solução e (2) usar indução matemática para provar que a conjectura está correta, determinando as constantes apropriadas.

4.1 Estrutura do Método

Suponha que desejamos resolver uma recorrência $T(n)$. O método procede da seguinte forma:

1. **Conjecturar** um limitante da forma $T(n) \leq c \cdot g(n)$ (para um limitante superior) ou $T(n) \geq c \cdot g(n)$ (para um limitante inferior), onde $g(n)$ é a função candidata e $c > 0$ é uma constante a ser determinada.
2. **Provar por indução** que a conjectura vale para todo n suficientemente grande, assumindo que ela vale para entradas menores.

4.2 Exemplo: Recorrência do Merge-Sort

Considere a recorrência

$$T(n) = 2\,T(\lfloor n/2 \rfloor) + dn$$

com $T(1) = \Theta(1)$ e $d > 0$ uma constante que reflete o custo linear do procedimento Merge. Conjecturamos que $T(n) = O(n \lg n)$, ou seja, que existe uma constante $c > 0$ tal que $T(n) \leq c\,n \lg n$ para todo $n \geq n_0$.

Passo indutivo. Assumimos, por indução forte, que $T(m) \leq c\,m \lg m$ para todo $m < n$. Em particular, $T(\lfloor n/2 \rfloor) \leq c\,\lfloor n/2 \rfloor \lg \lfloor n/2 \rfloor$. Substituindo na recorrência:

$$\begin{aligned}
T(n) &\leq 2\big(c\,\lfloor n/2 \rfloor \lg \lfloor n/2 \rfloor\big) + dn \\
&\leq c\,n \lg(n/2) + dn \\
&= c\,n \lg n - c\,n + dn \\
&= c\,n \lg n - (c-d)\,n \\
&\leq c\,n \lg n
\end{aligned}$$

onde a última desigualdade vale desde que $c \geq d$, pois nesse caso $(c-d)\,n \geq 0$.

Caso base. Seja $T(1) = a$ para alguma constante $a > 0$. Para $n = 1$, a conjectura exigiria $T(1) \leq c \cdot 1 \cdot \lg 1 = 0$, o que falha. Para $n = 2$, temos $T(2) = 2\,T(1) + 2d = 2a + 2d$. Precisamos que $T(2) \leq c \cdot 2 \lg 2 = 2c$, o que vale para $c \geq a + d$. Portanto, tomando $n_0 = 2$ e $c \geq a + d \geq d$, ambas as condições — do caso base e do passo indutivo — são satisfeitas simultaneamente.

Concluímos que $T(n) \leq c\,n \lg n$ para todo $n \geq 2$, e assim $T(n) = O(n \lg n)$.

4.3 Flexibilidade na Escolha do Caso Base

Ao aplicar o método de substituição, é comum que a conjectura $T(n) \leq c\,g(n)$ falhe para valores pequenos de n. Isso não invalida a prova, desde que a indução seja iniciada a partir de um n_0 adequado. O Cormen (CLRS) dedica atenção explícita a este ponto, pois ele é fonte frequente de confusão.

4.3.1 Por que a Falha em $n = 1$ Não É Problema

Recordemos que a notação assintótica $T(n) = O(g(n))$ exige apenas que $T(n) \leq c\,g(n)$ para todo $n \geq n_0$, onde tanto c quanto n_0 são constantes que podemos escolher. Não há obrigação de que a desigualdade valha para *todo* n; basta que valha a partir de algum ponto.

No exemplo da recorrência do Merge-Sort, a conjectura $T(n) \leq c\,n \lg n$ falha para $n = 1$ porque $\lg 1 = 0$, de modo que o lado direito se anula independentemente de c. Contudo, isso reflete apenas a inadequação da função $n \lg n$ como limitante para $n = 1$, e não uma falha da conjectura assintótica.

4.3.2 A Justificativa Formal

A prova por indução forte requer dois ingredientes: que $T(n) \leq c\,g(n)$ valha para todos os valores de n em um intervalo base $[n_0, \ldots, n_1]$ suficiente para ancorar a indução, e que o passo indutivo preserve a desigualdade para todo $n > n_1$. O intervalo base precisa cobrir todos os valores que a recorrência pode atingir antes de se reduzir a subproblemas já tratados pelo passo indutivo.

No caso de $T(n) = 2\,T(\lfloor n/2 \rfloor) + dn$, qualquer $n \geq 2$ gera subproblemas de tamanho $\lfloor n/2 \rfloor \geq 1$. Se tomamos $n_0 = 2$, o menor subproblema que o passo indutivo precisa consultar é $T(1)$, que não está coberto pela hipótese. Porém, como $T(1)$ é uma constante conhecida — digamos $T(1) = a$ — podemos incorporá-la diretamente na verificação do caso base para $n = 2$:

$$T(2) = 2a + 2d \leq 2c = c \cdot 2 \lg 2$$

que vale para $c \geq a + d$. A partir daí, todo $n \geq 3$ gera subproblemas de tamanho $\lfloor n/2 \rfloor \geq 2$, e o passo indutivo se aplica normalmente.

4.3.3 A Observação do Cormen

O Cormen formaliza esta flexibilidade da seguinte maneira: como a notação assintótica só exige que a desigualdade valha para $n \geq n_0$, podemos substituir as condições de contorno da recorrência — que tipicamente cobrem um número finito e constante de valores — pela escolha de c suficientemente grande para acomodá-las. Um número finito de condições da forma $T(n_i) \leq c\,g(n_i)$ pode sempre ser satisfeito tomando $c \geq \max_i\{T(n_i)/g(n_i)\}$, desde que $g(n_i) > 0$ para cada n_i no intervalo base.

Isso significa, na prática, que ao aplicar o método de substituição podemos nos concentrar no passo indutivo — que é onde a estrutura da recorrência é explorada — e tratar o caso base como uma verificação técnica, ajustando c e n_0 ao final da prova.

4.4 Armadilhas Comuns

O Cormen destaca algumas armadilhas frequentes ao aplicar o método de substituição:

A primeira é **errar na forma da hipótese indutiva**. Por exemplo, ao tentar provar que $T(n) = 2\,T(\lfloor n/2 \rfloor) + n$ é $O(n)$, assumimos $T(\lfloor n/2 \rfloor) \leq c\,\lfloor n/2 \rfloor$ e obtemos $T(n) \leq c\,n + n = (c+1)\,n$, o que *não* prova $T(n) \leq c\,n$ para a *mesma* constante c. A indução falha porque o termo residual n impede que a hipótese se "feche".

A segunda é a necessidade de, em certos casos, **subtrair um termo de ordem inferior** na hipótese indutiva para que a prova funcione. Por exemplo, para a recorrência $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$, a conjectura $T(n) \leq c\,n$ não fecha diretamente. No entanto, conjecturar $T(n) \leq c\,n - d$ para constantes $c, d > 0$ fornece a margem necessária para absorver o termo constante e completar a indução.

4.5 Como Obter Boas Conjecturas

O Cormen sugere algumas heurísticas para formular a conjectura inicial. Se a recorrência é semelhante a uma já conhecida, pode-se usar a mesma forma: por exemplo, $T(n) = 2\,T(\lfloor n/2 \rfloor + 17) + n$ provavelmente tem solução $O(n \lg n)$, pois o termo aditivo 17 torna-se insignificante para n grande. Outra estratégia é provar limitantes frouxos — primeiro $O(n^2)$ e $\Omega(n)$ — e ir refinando progressivamente até que os limitantes superior e inferior coincidam. Finalmente, pode-se usar a **árvore de recursão** (apresentada na próxima seção) como ferramenta para gerar uma boa conjectura, que depois é verificada rigorosamente pelo método de substituição.

5 Método da Árvore de Recursão

O **método da árvore de recursão** é uma técnica para resolver recorrências que consiste em expandir a recorrência em uma árvore cujos nós representam os custos incorridos em cada nível de recursão. Somando os custos de todos os níveis, obtém-se uma expressão para o custo total, que pode ser usada diretamente como solução ou como conjectura a

ser verificada rigorosamente pelo método de substituição. O Cormen (CLRS) apresenta a árvore de recursão tanto como ferramenta heurística para gerar boas conjecturas quanto, quando conduzida com cuidado suficiente, como prova direta.

5.1 Construção da Árvore

Considere uma recorrência da forma geral

$$T(n) = a\,T(n/b) + f(n)$$

com caso base $T(1) = \Theta(1)$. A árvore de recursão é construída da seguinte maneira:

1. A **raiz** contém o custo $f(n)$, correspondente ao trabalho realizado fora das chamadas recursivas.
2. Cada nó interno com custo relativo a um subproblema de tamanho m gera a filhos, cada um associado a um subproblema de tamanho m/b.
3. As **folhas** correspondem ao caso base, cada uma contribuindo com custo $\Theta(1)$.

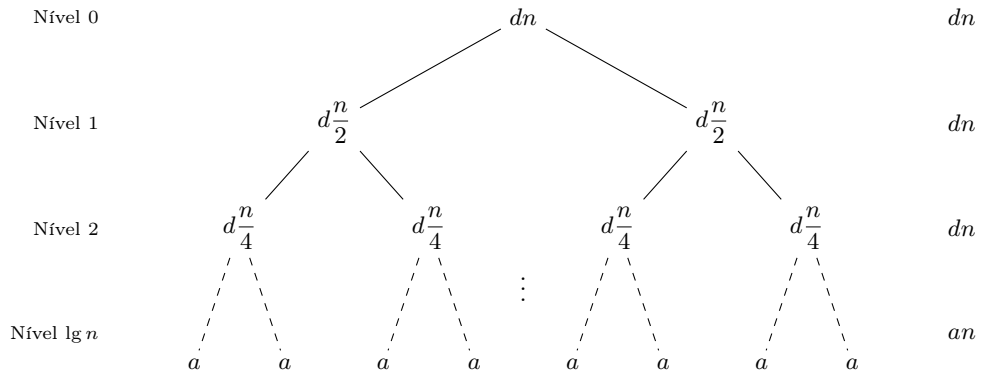
O custo total é então a soma dos custos de todos os nós, que pode ser organizada nível a nível.

5.2 Exemplo: Recorrência do Merge-Sort

Considere a recorrência

$$T(n) = 2\,T(n/2) + dn$$

com $T(1) = a$. A árvore tem a seguinte estrutura:



No nível i, há 2^i nós, cada um com custo $d(n/2^i)$, de modo que o custo total do nível é $2^i \cdot d(n/2^i) = dn$. A recursão atinge o caso base quando $n/2^i = 1$, isto é, quando $i = \lg n$. Portanto a árvore tem $\lg n + 1$ níveis (de 0 a $\lg n$). No último nível há $2^{\lg n} = n$ folhas, cada uma com custo a, totalizando an.

O custo total é a soma dos custos de todos os níveis:

$$T(n) = \underbrace{dn + dn + \cdots + dn}_{\lg n \text{ parcelas}} + an = dn \lg n + an$$

e portanto $T(n) = \Theta(n \lg n)$.

5.3 Exemplo: Recorrência com Subproblemas Desiguais

Considere agora a recorrência

$$T(n) = T(n/3) + T(2n/3) + dn$$

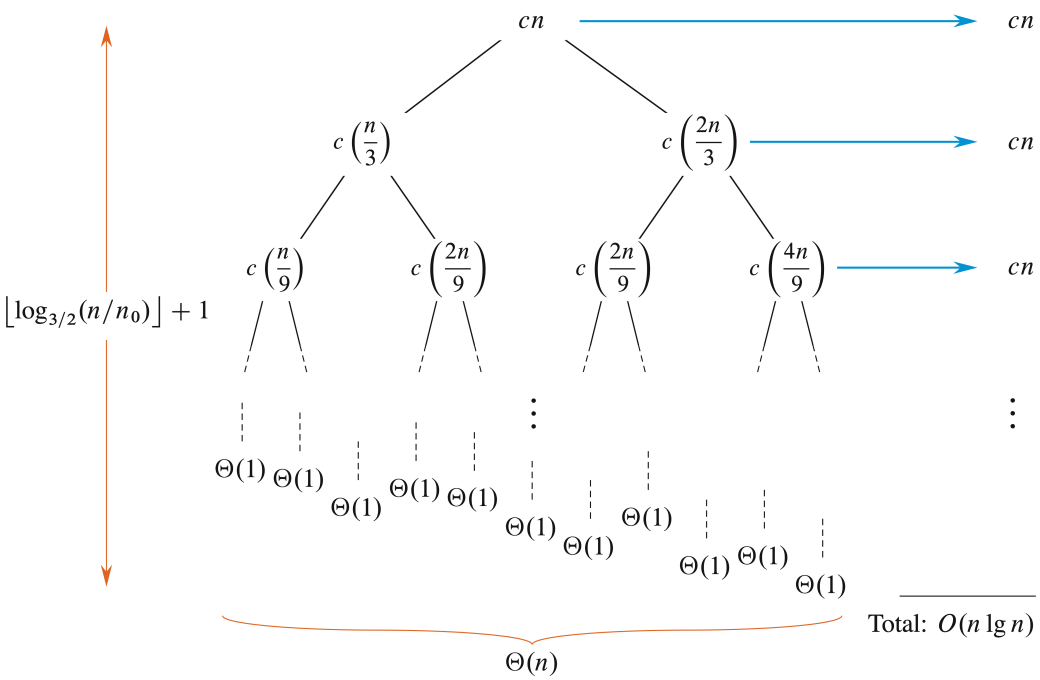
que surge quando um algoritmo de divisão e conquista particiona o problema de forma desbalanceada.

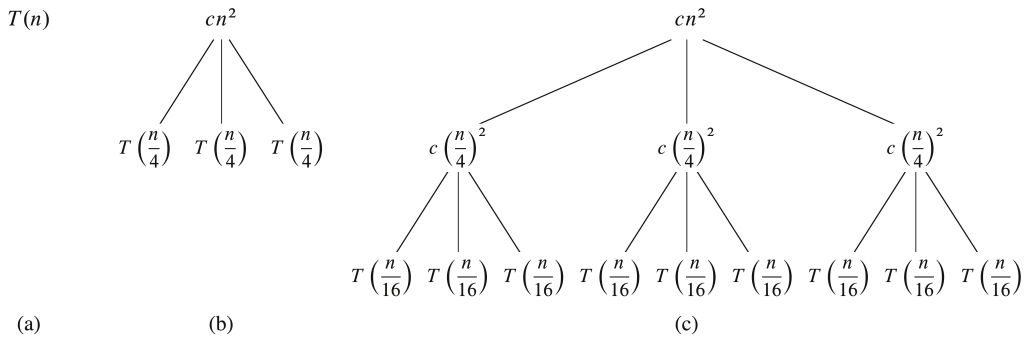
A árvore não é completa: o ramo esquerdo reduz o tamanho por um fator de 3, atingindo o caso base no nível $\log_3 n$, enquanto o ramo direito reduz por um fator de 3/2, atingindo o caso base no nível $\log_{3/2} n$. Apesar da assimetria, o custo de cada nível completo é no máximo dn, pois os tamanhos dos subproblemas em qualquer nível somam no máximo n. O caminho mais longo da raiz a uma folha tem comprimento $\log_{3/2} n$, de modo que a árvore tem no máximo $\log_{3/2} n + 1$ níveis. Portanto:

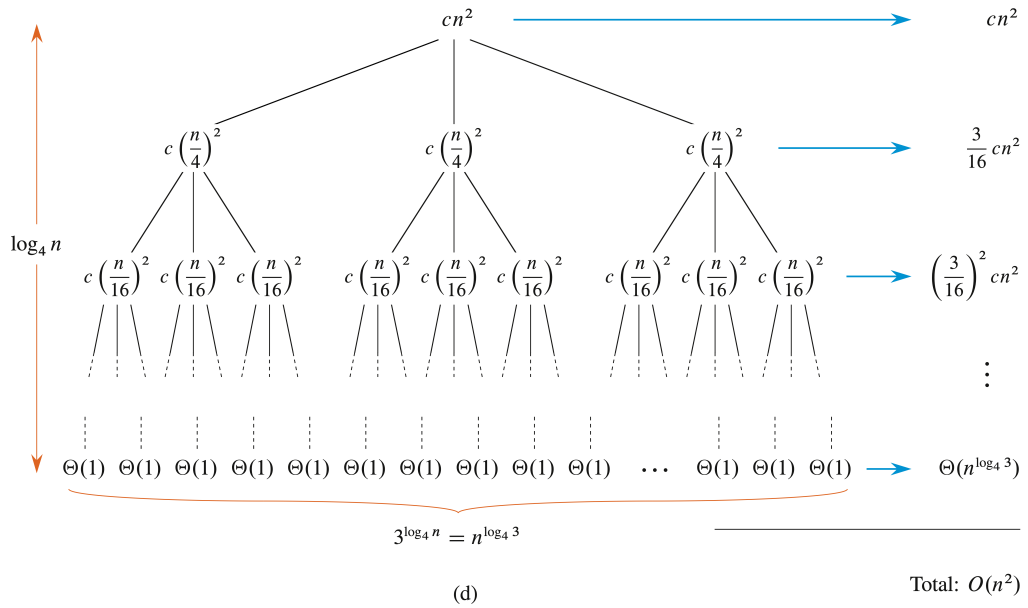
$$T(n) \leq dn \cdot (\log_{3/2} n + 1) = O(n \lg n)$$

onde usamos que $\log_{3/2} n = \Theta(\lg n)$, pois a mudança de base introduz apenas um fator constante. Para confirmar que $T(n) = \Omega(n \lg n)$, basta observar que o caminho mais curto tem comprimento $\log_3 n = \Theta(\lg n)$ e que cada um desses níveis contribui com custo exatamente dn. Concluímos que $T(n) = \Theta(n \lg n)$.

Como o Cormen observa, este é um caso em que a árvore de recursão é especialmente útil: a recorrência não se enquadra diretamente na forma $T(n) = a\,T(n/b) + f(n)$ com subproblemas de tamanho igual, mas a árvore fornece uma conjectura precisa que pode ser verificada pelo método de substituição.







5.4 Exemplo: Custo Geométrico por Nível

Considere a recorrência

$$T(n) = 3\,T(n/4) + dn^2$$

Neste caso, cada nível da árvore tem um custo diferente. No nível i, há 3^i nós, cada um com custo $d(n/4^i)^2$, de modo que

o custo total do nível é $3^i \cdot d(n/4^i)^2 = \left(\frac{3}{16}\right)^i dn^2$. A profundidade da árvore é $\log_4 n$, pois o caso base é atingido quando $n/4^i = 1$. A soma dos custos por nível forma uma série geométrica:

$$T(n) = dn^2 \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i + \Theta\left(n^{\log_4 3}\right)$$

onde o último termo corresponde ao custo das $3^{\log_4 n} = n^{\log_4 3}$ folhas. Como a razão $3/16 < 1$, a série geométrica converge, e obtemos:

$$\sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i < \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i = \frac{1}{1 - 3/16} = \frac{16}{13}$$

Portanto $T(n) \leq \frac{16}{13} dn^2 + \Theta(n^{\log_4 3}) = O(n^2)$, pois $\log_4 3 < 1 < 2$. O custo é dominado pelo nível da raiz, que é o cenário típico quando $f(n)$ cresce mais rápido que o número de subproblemas — uma situação que será formalizada pelo caso 3 do Teorema Mestre.

5.5 Árvore de Recursão como Conjectura

O Cormen enfatiza que a árvore de recursão é, em muitos casos, utilizada como ferramenta heurística: ela fornece uma conjectura para a forma assintótica de $T(n)$, que deve ser verificada rigorosamente pelo método de substituição. No entanto, quando a análise é conduzida com cuidado — somando corretamente todos os níveis e tratando pisos, tetos e casos base — a própria árvore constitui uma prova válida.

6 Método da Árvore de Recursão

O **método da árvore de recursão** é uma técnica para resolver recorrências que consiste em expandir a recorrência em uma árvore cujos nós representam os custos incorridos em cada nível de recursão. Somando os custos de todos os níveis, obtém-se uma expressão para o custo total, que pode ser usada diretamente como solução ou como conjectura a ser verificada rigorosamente pelo método de substituição. O Cormen (CLRS) apresenta a árvore de recursão tanto como ferramenta heurística para gerar boas conjecturas quanto, quando conduzida com cuidado suficiente, como prova direta.

6.1 Construção da Árvore

Considere uma recorrência da forma geral

$$T(n) = a\,T(n/b) + f(n)$$

com caso base $T(1) = \Theta(1)$. A árvore de recursão é construída da seguinte maneira:

1. A **raiz** contém o custo $f(n)$, correspondente ao trabalho realizado fora das chamadas recursivas.
2. Cada nó interno com custo relativo a um subproblema de tamanho m gera a filhos, cada um associado a um subproblema de tamanho m/b.
3. As **folhas** correspondem ao caso base, cada uma contribuindo com custo $\Theta(1)$.

O custo total é então a soma dos custos de todos os nós, que pode ser organizada nível a nível.

6.2 Exemplo: Recorrência do Merge-Sort

Considere a recorrência

$$T(n) = 2\,T(n/2) + dn$$

com $T(1) = a$. A árvore tem a seguinte estrutura:

- **Nível 0:** um único nó com custo dn. Há $1 = 2^0$ subproblema de tamanho $n/2^0 = n$.
- **Nível 1:** dois nós, cada um com custo $d(n/2)$. O custo total do nível é $2 \cdot d(n/2) = dn$.
- **Nível 2:** quatro nós, cada um com custo $d(n/4)$. O custo total do nível é $4 \cdot d(n/4) = dn$.
- **Nível i:** há 2^i nós, cada um com custo $d(n/2^i)$. O custo total do nível é $2^i \cdot d(n/2^i) = dn$.

A recursão atinge o caso base quando $n/2^i = 1$, isto é, quando $i = \lg n$. Portanto a árvore tem $\lg n + 1$ níveis (de 0 a $\lg n$). No último nível há $2^{\lg n} = n$ folhas, cada uma com custo a, totalizando an.

O custo total é a soma dos custos de todos os níveis:

$$T(n) = \underbrace{dn + dn + \cdots + dn}_{\lg n \text{ parcelas}} + an = dn \lg n + an$$

e portanto $T(n) = \Theta(n \lg n)$.

6.3 Exemplo: Recorrência com Subproblemas Desiguais

Considere agora a recorrência

$$T(n) = T(n/3) + T(2n/3) + dn$$

que surge quando um algoritmo de divisão e conquista particiona o problema de forma desbalanceada.

Neste caso, a árvore não é completa: o ramo esquerdo reduz o tamanho por um fator de 3, atingindo o caso base no nível $\log_3 n$, enquanto o ramo direito reduz por um fator de 3/2, atingindo o caso base no nível $\log_{3/2} n$. Apesar da assimetria, o custo de cada nível completo é no máximo dn, pois os tamanhos dos subproblemas em qualquer nível somam no máximo n. O caminho mais longo da raiz a uma folha tem comprimento $\log_{3/2} n$, de modo que a árvore tem no máximo $\log_{3/2} n + 1$ níveis. Portanto:

$$T(n) \leq dn \cdot (\log_{3/2} n + 1) = O(n \lg n)$$

onde usamos que $\log_{3/2} n = \Theta(\lg n)$, pois a mudança de base introduz apenas um fator constante. Para confirmar que $T(n) = \Omega(n \lg n)$, basta observar que o caminho mais curto tem comprimento $\log_3 n = \Theta(\lg n)$ e que cada um desses níveis contribui com custo exatamente dn. Concluímos que $T(n) = \Theta(n \lg n)$.

Como o Cormen observa, este é um caso em que a árvore de recursão é especialmente útil: a recorrência não se enquadra diretamente na forma $T(n) = a\,T(n/b) + f(n)$ com subproblemas de tamanho igual, mas a árvore fornece uma conjectura precisa que pode ser verificada pelo método de substituição.

6.4 Exemplo: Custo Geométrico por Nível

Considere a recorrência

$$T(n) = 3\,T(n/4) + dn^2$$

Neste caso, cada nível da árvore tem um custo diferente:

- **Nível 0:** custo dn^2.
- **Nível 1:** 3 nós, cada um com custo $d(n/4)^2$. Total: $3 \cdot d(n/4)^2 = \frac{3}{16}\,dn^2$.
- **Nível i:** 3^i nós, cada um com custo $d(n/4^i)^2$. Total: $3^i \cdot d(n/4^i)^2 = \left(\frac{3}{16}\right)^i dn^2$.

A profundidade da árvore é $\log_4 n$, pois o caso base é atingido quando $n/4^i = 1$. A soma dos custos por nível forma uma série geométrica:

$$T(n) = dn^2 \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i + \Theta\left(n^{\log_4 3}\right)$$

onde o último termo corresponde ao custo das $3^{\log_4 n} = n^{\log_4 3}$ folhas. Como a razão $3/16 < 1$, a série geométrica converge, e obtemos:

$$\sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i < \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i = \frac{1}{1 - 3/16} = \frac{16}{13}$$

Portanto $T(n) \leq \frac{16}{13}\,dn^2 + \Theta(n^{\log_4 3}) = O(n^2)$, pois $\log_4 3 < 1 < 2$. O custo é dominado pelo nível da raiz, que é o cenário típico quando $f(n)$ cresce mais rápido que o número de subproblemas — uma situação que será formalizada pelo caso 3 do Teorema Mestre.

6.5 Árvore de Recursão como Conjectura

O Cormen enfatiza que a árvore de recursão é, em muitos casos, utilizada como ferramenta heurística: ela fornece uma conjectura para a forma assintótica de $T(n)$, que deve ser verificada rigorosamente pelo método de substituição. No entanto, quando a análise é conduzida com cuidado — somando corretamente todos os níveis e tratando pisos, tetos e casos base — a própria árvore constitui uma prova válida.

7 Método mestre

Referências

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Algoritmos: teoria e prática*. Grupo GEN | LTC, Rio de Janeiro, 4 edition.